Universidad Diego Portales — Sistemas Operativos

Tarea 2: Komikan Concurrente

Maximiliano Santibañez y Benjamín Zuñiga

1. Cómo protegemos el tablero

El programa usa hilos POSIX en C++. Cada mesa tiene un hilo que hace de juez y 13 hilos para las piezas: uno para el puma y 12 para los perros. Las piezas no tienen turnos. Cada una espera un tiempo al azar, entre 50 y 300 ms por defecto, y luego intenta moverse.

Para cuidar el tablero usamos un mutex por mesa. Un mutex funciona como un candado: solo un hilo puede tenerlo a la vez. Este candado protege las casillas, la posición de las piezas, el número de capturas y los datos que indican si la partida terminó y quién ganó.

Cuando una ficha intenta moverse, primero toma el candado. Después revisa si el movimiento es válido y si el destino está libre. Si puede moverse, actualiza los datos y suelta el candado. En una captura también se retira al perro y se revisa si alguien ganó antes de soltarlo. Así nadie puede leer o cambiar el tablero cuando un movimiento está a medio hacer.

Por ejemplo, si dos perros quieren llegar a la misma casilla, el primero que toma el candado y cumple las reglas la ocupa. El segundo encuentra la casilla ocupada y no se mueve. Las condiciones se revisan dentro del candado, aunque la ficha haya elegido su destino antes. El juez también usa el candado para leer el tablero.

El servidor tiene otro mutex para la fila de jugadores y el número de mesas ocupadas. Si falta una pareja o no hay mesa libre, espera usando variables de condición. Estas liberan el mutex mientras se espera, para que los demás hilos puedan trabajar. Al despertar, se revisa otra vez si hay jugadores o cupo. También hay un mutex para que los mensajes de distintas mesas no se mezclen. Al terminar una partida, se espera a que salgan sus hilos antes de destruir el mutex de la mesa.

2. Qué problemas podrían aparecer

Deadlock: hilos que quedan esperando entre sí. Esto podría pasar si cada casilla tuviera un candado y dos fichas los tomaran en distinto orden. Usamos uno solo por mesa para evitar ese problema. Las funciones de movimiento siempre lo sueltan al terminar y no intentan tomarlo dos veces seguidas sin liberarlo. Además, el mutex del servidor se libera antes de iniciar una mesa, y el de los mensajes se toma después de soltar el del tablero.

Livelock: muchos intentos, pero ningún avance. Las fichas podrían elegir varias veces casillas ocupadas y fallar una y otra vez. Las pausas y las elecciones al azar ayudan a reducir estas repeticiones, aunque no aseguran que siempre haya un movimiento válido. El juez revisa un límite de 10 segundos y cierra la partida sin ganador si se supera. Esto limita las partidas que no avanzan, pero no arreglaría un candado que quedara tomado por error.

Starvation: una ficha queda sin oportunidad. Una ficha podría perder muchas veces la competencia por el mutex o por una casilla. Para reducir ese riesgo, mantenemos corto el tiempo dentro del candado y hacemos las pausas fuera de él. Aun así, el mutex no asegura que todos entren en orden, por lo que no podemos garantizar que todas las fichas logren moverse. En el servidor, los jugadores se emparejan en el orden en que entran a la fila.

3. Por qué elegimos este diseño

Elegimos un mutex por mesa porque el tablero es pequeño y resulta más fácil revisar que los datos queden correctos. Usar un candado por casilla permitiría hacer más movimientos a la vez, pero complicaría las capturas y la revisión de si el puma está encerrado.

La desventaja es que, dentro de una mesa, los movimientos se aplican de uno en uno, incluso si las fichas están lejos. Esto limita la cantidad de movimientos por segundo (*throughput*) y puede aumentar el tiempo de espera de un movimiento (latencia). De todas formas, las distintas mesas pueden avanzar al mismo tiempo porque cada una tiene su propio mutex. Si se reducen mucho las pausas o hay muchas mesas imprimiendo mensajes, puede aumentar la espera.

No medimos throughput ni latencia, así que esta comparación se basa en cómo funciona el código. El archivo `test_mesa.cpp` incluye una prueba de 1000 rondas donde cuatro perros intentan ocupar la misma casilla y se revisa que solo uno lo consiga. Esta prueba ayuda a comprobar la exclusión mutua, pero no demuestra que todas las fichas tengan las mismas oportunidades.